

MACHINE LEARNING

(ITY23PCL302)

UNIT 2 FEATURE SELECTION

Course Teacher: Ms.Priyanka A.Hajare

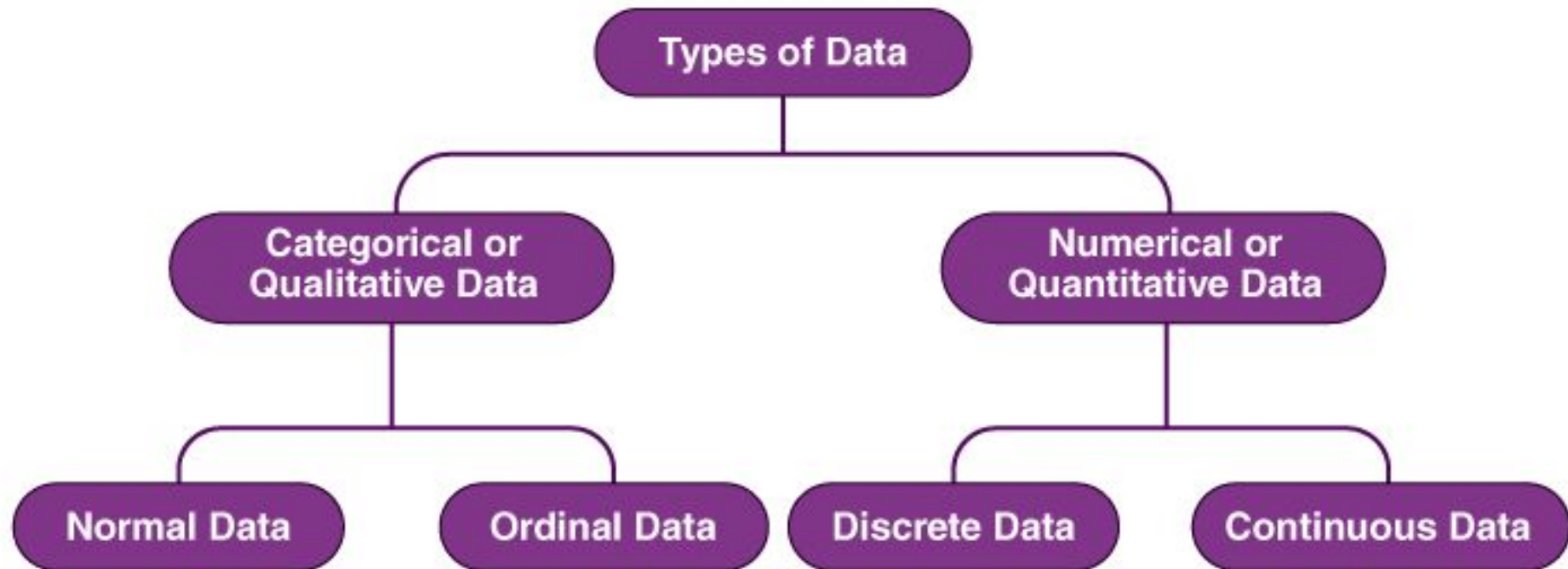
CONTENT

2	Feature Selection - Scikit-learn Dataset, Creating training and test sets, managing categorical data, Managing missing features, Data scaling and normalization, Feature selection and Filtering, Principal Component Analysis(PCA)-non negative matrix factorization, Sparse PCA, Kernel PCA. Atom Extraction and Dictionary Learning	6
	Regression - Linear regression, Ridge regression, Lasso regression, Elastic net	

Course Outcomes

CO1	Distinguish different learning based applications. (BTL2 : Understand)
CO2	To apply different processing methods to prepare training data set for Machine Learning. (BTL3 : Apply)
CO3	To design and implement supervised and unsupervised machine learning algorithm.(BTL3 : Apply)(BTL6 : Create)
CO4	To implement different learning models.(BTL3 : Apply)
CO5	Learn Meta classifiers and deep learning concepts. (BTL2 : Understand)

Types of Data



Qualitative or Categorical Data

- Qualitative or Categorical Data is data that can't be measured or counted in the form of numbers.
- These types of data are sorted by category, not by number.
- These data consist of audio, images, symbols, or text.

Examples

- language do you speak
- Favorite holiday destination
- Opinion on something (agree, disagree, or neutral)
- Colors

Nominal Data

- Nominal Data is used to label variables without any order or quantitative value.
- With the help of nominal data, we can't do any numerical tasks or can't give any order to sort the data.
- These data don't have any meaningful order; their values are distributed into distinct categories.
- Colour of hair (Blonde, red, Brown, Black, etc.)
- Marital status (Single, Widowed, Married)
- Nationality (Indian, German, American)
- Gender (Male, Female, Others)
- Eye Color (Black, Brown, etc.)

Ordinal Data

- Ordinal data have natural ordering where a number is present in some kind of order by their position on the scale.
- These data are used for observation like customer satisfaction, happiness, etc., but we can't do any arithmetic tasks on them.

Examples of Ordinal Data :

- When companies ask for feedback, experience, or satisfaction on a scale of 1 to 10
- Letter grades in the exam (A, B, C, D, etc.)
- Ranking of people in a competition (First, Second, Third, etc.)
- Economic Status (High, Medium, and Low)
- Education Level (Higher, Secondary, Primary)

Quantitative Data

- Quantitative data can be expressed in numerical values, making it countable and including statistical data analysis.
- These kinds of data are also known as Numerical data.

Examples of Quantitative Data :

- Height or weight of a person or object
- Room Temperature
- Scores and Marks (Ex: 59, 80, 60, etc.)
- Time

Discrete Data

Examples of Discrete Data :

- Total numbers of students present in a class
- Cost of a cell phone
- Numbers of employees in a company
- The total number of players who participated in a competition
- Days in a week

Continuous Data

Examples of Continuous Data :

- Height of a person
- Speed of a vehicle
- “Time-taken” to finish the work
- Wi-Fi Frequency
- Market share price

Identify the type of Data?

- Profession(Doctor, Engineer, Teacher)
- The first, second and third person in a competition.
- Religion (Muslin, Buddhist, Christian)
- Economic status: low, medium and high.
- Education(Primary, Secondary, Graduation)

Feature Selection

- **Feature selection in machine learning is the process of choosing a subset of relevant features** from the original set of features to build a more effective and efficient model.
- When a dataset has too many features, some may be irrelevant or add noise which can slow down training process and reduce accuracy but it helps in building simpler, faster and more accurate models and also helps in reducing overfitting.

The benefits of feature selection

- **Better model performance:** Irrelevant features weaken model performance. Conversely, choosing the right set of features for a model makes it more accurate, more precise and gives it better recall. Data features affect how models configure their weights during training, which in turn drives performance. This differs from [hyperparameter tuning](#), which occurs before training.
- **Reduced overfitting:** [Overfitting](#) happens when a model cannot generalize past its training data. Removing redundant features decreases overfitting and makes a model better able to generalize to new data.
- **Shorter training times:** By focusing on a smaller subset of features, algorithms take less time to train. Model creators can test, validate and deploy their models faster with a smaller set of selected features.
- **Lower compute costs:** A smaller dataset made of the best features makes for simpler [predictive models](#) that occupy less storage space. Their computational requirements are lower than those of more complex models.
- **Greater interpretability:** [Explainable AI](#) is focused on creating models that humans can understand. As models grow more complex, it becomes increasingly difficult to [interpret](#) their results. Simpler models are easier to monitor and explain.
- **Smoother implementation:** Simpler, smaller models are easier to work with by developers when building AI apps, such as those used in [data visualization](#).
- **Dimensionality reduction:** With more input variables in play, data points grow more distant within the model space. High-dimensional data has more empty space, which makes it more difficult for the machine learning algorithm to identify patterns and make good predictions.
- Collecting more data can mitigate the curse of dimensionality, but selecting the most important features is more feasible and cost-effective.
- Ref: <https://www.ibm.com/think/topics/feature-selection>

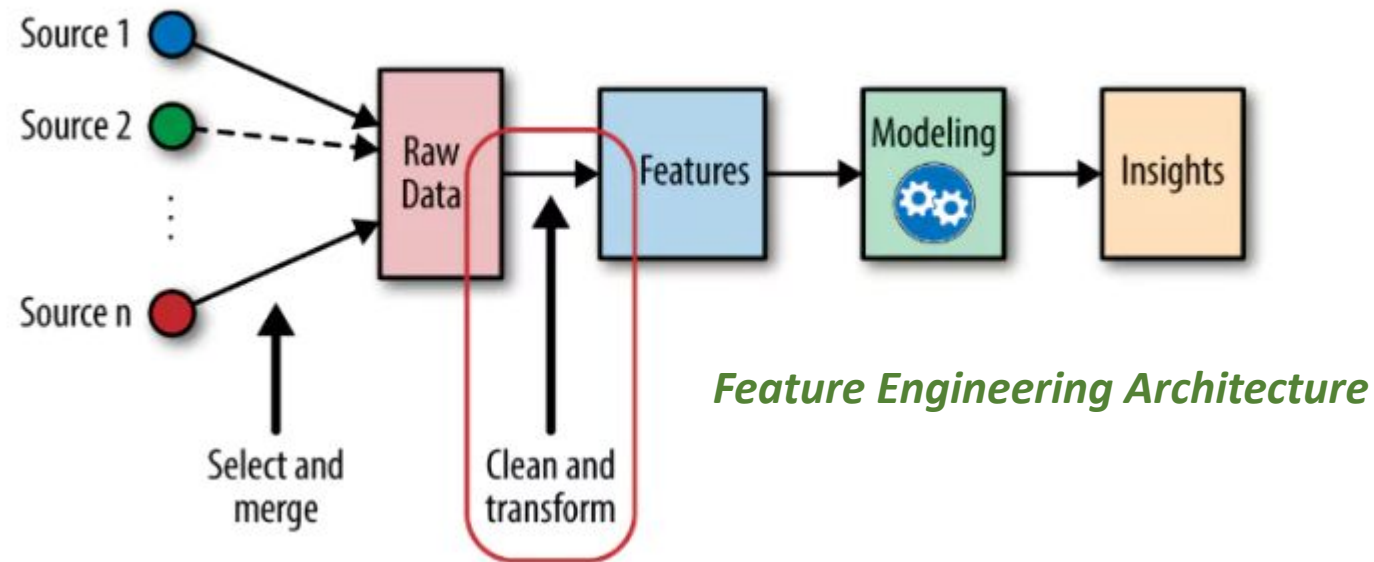
- **BEFORE FEATURE SELECTION AND TECHNIQUES DO YOU HAVE IDEA ABOUT FOLLOWING WORDS ????**

What are Features?

- Features are **the individual measurable properties or characteristics of the data** that are used by a model to make predictions or decisions. **Feature Vector is A collection of features represented in a structured form.**
- Types of feature :
 - **Numerical:** Continuous values (e.g., height, salary)
 - **Categorical:** Discrete labels (e.g., color, city name)
 - **Ordinal:** Ordered categories (e.g., rating: low, medium, high)
 - **Boolean:** Binary values (e.g., yes/no, true/false)
 - **Text/Images:** Can be converted into numerical representations using feature extraction techniques (like TF-IDF or pixel intensities)
 - **time-series features :**time-series features represent timely data, such as weekly sales or stock price fluctuations over a year.

What is Feature Engineering?

- Feature engineering is the process of selecting, transforming, extracting, combining, and manipulating raw data to generate the desired variables for analysis or predictive modeling. It is a crucial step in developing a machine learning model.



Processes Involved in Feature Engineering

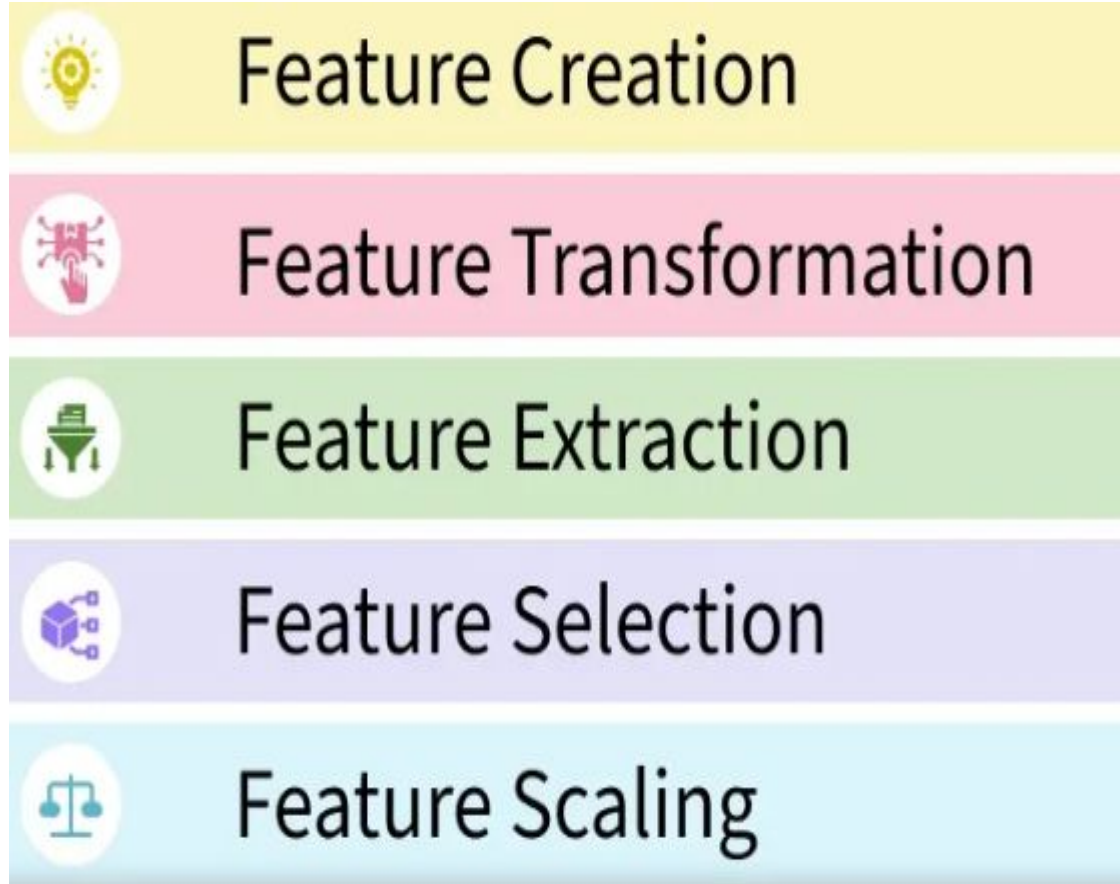


IMAGE REF :<https://www.geeksforgeeks.org/>

1. feature creation involves **generating new features** from domain knowledge or by observing patterns in the data
2. Transformation adjusts features to improve model learning **Normalization & Scaling , Encoding, Mathematical transformations.**
3. **Feature Extraction:** Extracting meaningful features can reduce dimensionality and improve model accuracy.
4. **Feature Selection:** Feature selection involves choosing a subset of relevant features to use.
5. **Feature Scaling:** Scaling ensures that all features contribute equally to the model like **Min-Max scaling , Standard scaling.**

Feature Engineering Techniques

- **Handling Missing Values**
 - Imputation (mean, median, mode)
 - Deletion of rows/columns
 - Predictive imputation
- **Encoding Categorical Variables**
 - Label Encoding
 - One-Hot Encoding
- **Feature Scaling / Normalization**
 - Min-Max Scaling
 - Standardization (Z-score)
- **Outlier Handling**
 - Clipping
 - Capping/Flooring
- **Feature Construction / Creation**
 - Binning/Bucketing
 - Aggregation (e.g., average by group)
- **Feature Transformation**
 - Log Transformation
 - Rank Transformation
- **Dimensionality Reduction**
 - Principal Component Analysis (PCA)
- **Text Feature Extraction**
 - Bag of Words (BoW)
- **Date/Time Feature Engineering**
 - Extracting components (hour, day, month, etc.)
 - Calculating durations (e.g., time since event)

Which one is right method

- Choice of feature selection method depends on several factors:
- **Dataset Size:** Filter methods are often preferred for very large datasets due to their speed.
- **Feature Interactions:** Wrapper and embedded methods are better for capturing complex feature interactions.
- **Model Type:** Some methods like Lasso and decision trees are more suitable for certain models like linear models or tree-based models.

Scikit-learn Datasets

- scikit-learn provides access to **standard toy datasets** as well as **real-world datasets** that are commonly used for benchmarking ML algorithms.
- https://scikit-learn.org/stable/datasets/toy_dataset.html

Types of Datasets in Scikit-learn

Dataset Type	Examples	Description
Toy Datasets (small)	<code>load_iris()</code> , <code>load_digits()</code> , <code>load_wine()</code>	Small, easy to understand, good for practice
Real-world Datasets (medium-large)	<code>fetch_20newsgroups()</code> , <code>fetch_california_housing()</code>	Larger, real-world data; often require internet
Generated Datasets	<code>make_classification()</code> , <code>make_regression()</code>	Synthetic data for experimentation and benchmarking

Common Datasets in Scikit-learn

Dataset Name	Function	Type	Task	Features
Iris	load_iris()	Toy	Classification	4
Wine	load_wine()	Toy	Classification	13
Breast Cancer	load_breast_cancer()	Toy	Classification	30
Digits	load_digits()	Toy	Classification	64
California Housing	fetch_california_housing()	Real	Regression	8
20 Newsgroups	fetch_20newsgroups()	Real	Text Classification	N/A

How to Access Datasets

Let's do in real time .

Training and Test Data Set

- Train/Test is a method to measure the accuracy of your model.
- It is called Train/Test because data set is split into two sets: **a training set and a testing set.**
- **80% for training, and 20% for testing.**
- *train* the model using the training set. *Train* the model means *create* the model.
- *test* the model using the testing set. *Test* the model means test the accuracy of the model.

sklearn.datasets

- Toy Datasets (Small, In-Memory)
- Real-World Datasets (Larger, Downloadable)
- Synthetic Dataset Generators

Function	Description	Function	Description
		load_iris()	Classic iris flower classification dataset.
		load_boston() 	(Deprecated) Boston housing dataset.
		load_diabetes()	Dataset for regression tasks.
		load_digits()	Handwritten digits (0–9) dataset.
make_classification()	Creates a multi-class classification dataset.	load_wine()	Wine classification dataset.
make_regression()	Generates a regression problem dataset.	load_breast_cancer()	Breast cancer diagnostic dataset.
make_blobs()	Creates Gaussian blobs for clustering.		
make_moons()	Generates two interleaving half circles.		
make_circles()	Generates a large circle containing a smaller circle.		

Managing Categorical Data

- Categorical data are features that represent **discrete categories** instead of continuous values.
- Ex :
- Gender: Male, Female (Nominal Data)
- Education Level :High School < Bachelors < Masters < PhD (ordinal)

Process to Manage Categorical Data

Step	Action	Purpose
1	Identify Categorical Variables	Detect columns with object/string or category type
2	Understand Category Type	Determine if categories are nominal (no order) or ordinal (ordered)
3	Handle Missing Values	Fill or remove missing categorical entries
4	Encode the Variables	Use encoding techniques: One-Hot, Label, or others
5	Combine with Numeric Data	Join encoded features with existing numeric features
6	Feature Selection (optional)	Use statistical techniques to keep important features
7	Scale if Needed	Not always needed for categorical, but useful when combining with continuous variables
8	Use in ML Models	Feed processed data into machine learning algorithms

Encoding Techniques

- Encoding techniques in Machine Learning (ML) are used to convert **categorical variables** into **numerical format** so that algorithms can interpret them.
- Need :Most ML models require numerical input. Algorithms like linear regression, SVM, or neural networks **can't handle strings or categories** directly. Encoding solves this.

Types of Encoding Techniques in ML

Encoding Technique	Best For	Handles Ordinal?	Handles Nominal?	Library Support
Label Encoding	Ordinal Data	✓ Yes	Risky	sklearn.preprocessing
One-Hot Encoding	Nominal Data	✗ No	✓ Yes	pandas, sklearn
Ordinal Encoding	Ordinal Data	✓ Yes	✗ No	sklearn.preprocessing
Binary Encoding	High-cardinality	Partially	✓ Yes	category_encoders
Target/Mean Encoding	Supervised Tasks	Risk of leakage	✓ Yes	category_encoders
Frequency Encoding	High-cardinality	✗ No	✓ Yes	Custom / category_encoders
Hash Encoding	High-cardinality	✗ No	✓ Yes	category_encoders
BaseN Encoding	High-cardinality	✗ No	✓ Yes	category_encoders

Example

Consider data set shown below and how different encoding technique work to convert categorical data into numeric data.

	Gender	Education	City	Purchased
0	Male	High School	Delhi	0
1	Female	Bachelors	Mumbai	1
2	Female	Masters	Delhi	1
3	Male	PhD	Chennai	0
4	Female	Bachelors	Mumbai	1

Encoding Technique	Description
Label Encoding	Replaces each category with a unique number .
One-Hot Encoding	Creates separate columns for each category.
Ordinal Encoding	Assigns numbers based on order .
Target/Mean Encoding	Replaces a category with the mean target value for that category. Useful for high-cardinality data (e.g. hundreds of cities).

Encoding

Encoding Technique	Description
Binary encoding	combines the properties of One-Hot Encoding and Hashing . It's useful for high-cardinality categorical features (i.e., columns with many unique categories).
Frequency Encoding	Replaces each category with the frequency (count) of that category in the column.
Hash Encoding	Transforms each category into a fixed number of columns using hash functions. Hashing Encoding is a technique used to convert categorical variables into numerical form using a hash function . It's especially useful for high-cardinality features (like city names, product IDs, etc.) where One-Hot Encoding would result in too many columns .
BaseN Encoding	Encodes categories using numbers in a base-N system (e.g., base 3 or base 5).

Exercise

- **Problem Statement: Predicting Customer Purchase Behavior**
- **Context:** An e-commerce company wants to build a machine learning model to **predict whether a customer will purchase a product or not** based on demographic and behavioral data. The dataset contains a mix of **nominal, ordinal, and high-cardinality categorical features**, making it an ideal case to understand **different encoding techniques**.

Dataset to be consider

CustomerID	Gender	City	Education	Product_Cat egory	Age_Group	Purchased
101	Male	Mumbai	Bachelors	Electronics	26-35	1
102	Female	Delhi	Masters	Clothing	18-25	0
103	Female	Pune	High School	Clothing	36-45	1
104	Male	Bangalore	PhD	Groceries	26-35	0
105	Male	Delhi	Bachelors	Electronics	46-55	1

YOU CAN ADD FIELD AND ROWS AS PER YOUR REQUIREMENT

Feature Type Breakdown

Feature	Type of Categorical Data	Encoding Technique Suggested
Gender	Nominal (binary)	Label Encoding
City	Nominal (high cardinality)	Target Encoding / Frequency Encoding
Education	Ordinal	Ordinal Encoding
Product_Category	Nominal (low cardinality)	One-Hot Encoding
Age_Group	Ordered Ranges	Ordinal Encoding / Custom Mapping

ML Task

- Build a classification model to **predict the Purchased column** using encoded features.

Learning Goals

- Through this problem, you will learn :
 - What You Will Learn
 - How to distinguish nominal vs ordinal vs high-cardinality
 - How to choose the right encoding method per feature
 - How encoding impacts model performance
 - Hands-on with LabelEncoder, OneHotEncoder, OrdinalEncoder, and Target Encoding

Managing Missing Features

- In data science and machine learning, dealing with missing values is a critical step to ensure accurate and reliable model.
- Missing data is defined as the values or data that is not stored (or not present) for some variable/s in the given dataset. Below is a sample of the missing data from the Titanic dataset. You can see the columns 'Age' and 'Cabin' have some missing values.

<https://www.kaggle.com/code/alexisbcook/missing-values/tutorial>



PassengerId	Survived	Pclass	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	3	male	22	1	0	A/5 21171	7.25		S
2	1	1	female	38	1	0	PC 17599	71.2833	C85	C
3	1	3	female	26	0	0	STON/O2. 3101282	7.925		S
4	1	1	female	35	1	0	113803	53.1	C123	S
5	0	3	male	35	0	0	373450	8.05		S
6	0	3	male		0	0	330877	8.4583		Q

How is a Missing Value Represented in a Dataset?

- **NaN (Not a Number):** In many programming languages and data analysis tools, missing values are represented as NaN. This is the default for libraries like [Pandas](#) in Python.
- **NULL or None:** In databases and some programming languages, missing values are often represented as NULL or None. For instance, in [SQL databases](#), a missing value is typically recorded as NULL.
- **Empty Strings:** Sometimes, missing values are denoted by empty strings (""). This is common in text-based data or CSV files where a field might be left blank.
- **Special Indicators:** Datasets might use specific indicators like -999, 9999, or other unlikely values to signify missing data. This is often seen in older datasets or specific industries where such conventions were established.
- **Blanks or Spaces:** In some cases, particularly in fixed-width text files, missing values might be represented by spaces or blank fields.

Types of Missing Values

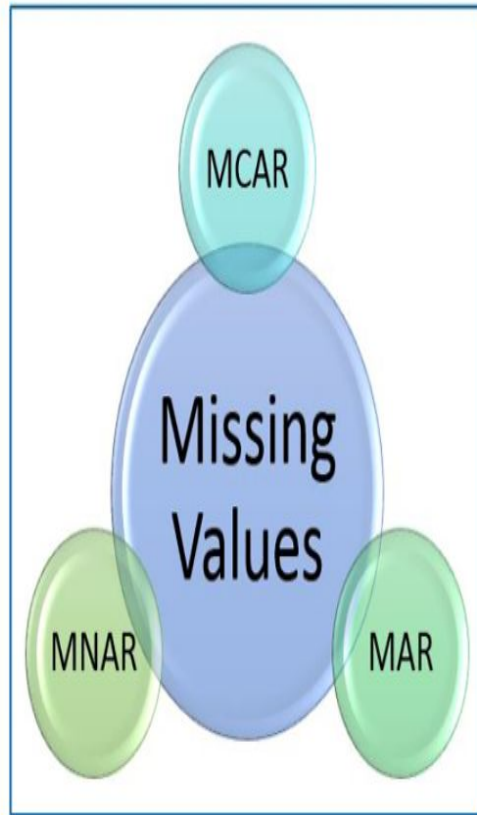


Figure 1 - Different Types of Missing Values in Datasets

Missing Completely At Random (MCAR)

MCAR occurs when the probability of data being missing is uniform across all observations. There is no relationship between the missingness of data and any other observed or unobserved data within the dataset.

Example: In a survey about library books, some overdue book values in the dataset are missing due to human error in recording.

Types of Missing Values

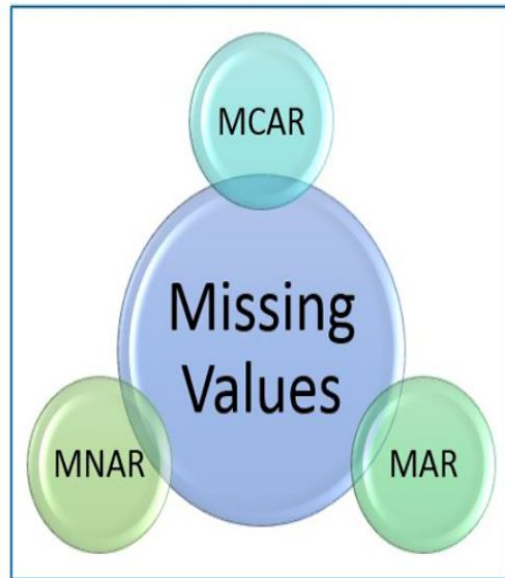


Figure 1 - Different Types of Missing Values in Datasets

Missing At Random (MAR)

MAR data occurs when the probability of data being missing depends only on the observed data and not on the missing data itself. In other words, the missingness can be explained by variables for which you have complete information. There is a pattern in the missing values, but this pattern can be explained by other observed variables.

Example: In a survey, 'Age' values might be missing for those who did not disclose their 'Gender'. Here, the missingness of 'Age' depends on 'Gender', but the missing 'Age' values are random among those who did not disclose their 'Gender'.

Missing Not At Random (MNAR)

MNAR occurs when the missingness of data is related to the unobserved data itself, which is not included in the dataset. This type of missing data has a specific pattern that cannot be explained by observed variables.

Example: In a survey about library books, people with more overdue books might be less likely to respond to the survey. Thus, the number of overdue books is missing and depends on the number of books overdue.

Normalization and Scaling

- In machine learning, the foundation for successful models is built on the quality of data they are trained on.
- But number of time **features differ in range and unit, hence affecting their importance.**
- **Scaling and normalization** prepare data to ensure fair treatment of all features in model training.
- These techniques are **essential preprocessing steps** in pipelines, especially before feature selection **These methods help rescale, standardize, or normalize feature values by adjusting their distribution and range**, thereby enhancing model accuracy and overall performance.

Normalization

- It's the process of casting the data to the specific range, like between 0 and 1 or between -1 and +1.
- Normalization is required when there are big differences in the ranges of different features.
- This is achieved by adjusting each feature values based on its minimum and maximum values.
- Normalization is the process of transforming data so that **all features are on a similar scale**.
- The goal is to ensure that no single feature dominates the others due to its magnitude.

Ref https://www.researchgate.net/publication/340579135_Data_Normalization_and_Standardization_A_Technical_Report [accessed Jul 26 2025].

When and Why to Use It

Algorithm Sensitivity: Essential for algorithms that rely on distance metrics, such as k-Nearest Neighbors (k-NN) and Support Vector Machines (SVM). It ensures that one feature doesn't dominate the distance calculation.

Neural Networks: Highly recommended for image processing or neural networks where inputs need to be bounded (e.g., pixel intensities from 0 to 255 scaled to 0 to 1) to match the activation functions like Sigmoid.

Distribution Shape: Normalization *preserves* the exact shape of your original distribution. It compressed the variance but does not alter the relative spacing or handle outliers well.

Mathematical Formula

$$X_{\text{norm}} = \frac{X - X_{\text{min}}}{X_{\text{max}} - X_{\text{min}}}$$

Where X_{min} and X_{max} are the minimum and maximum values of that feature in the dataset.

Standardization (Z-Score Scaling)

Standardization transforms your data so that it has a **mean (μ) of 0** and a **standard deviation (σ) of 1**. Unlike normalization, it does not bound the values to a specific range.

$$z = \frac{x - \mu}{\sigma}$$

Where μ is the feature mean: $\mu = \frac{1}{n} \sum_{i=1}^n x_i$

Where σ is the standard deviation: $\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2}$

When and Why to Use It

- **Optimization Algorithms:** Crucial for algorithms that use **Gradient Descent (e.g., Linear Regression, Logistic Regression, Neural Networks)** because it creates a **symmetric error surface**, allowing the gradient to converge much faster.
- **Dimensionality Reduction:** Required for **Principal Component Analysis (PCA)** because PCA maximizes variance; if one feature naturally has a massive variance due to its scale, PCA will mistakenly align all principal components with it.
- **Outlier Resilience:** **It handles outliers much better than Min-Max scaling.** While outliers still affect the mean and standard deviation, they won't crush the rest of the normal data into a tiny fractional range.

Why Scaling/Normalization Matter in ML

1. Better Model Accuracy

- Algorithms like **KNN, SVM, Logistic Regression, and PCA** are distance-based or gradient-based — thus sensitive to feature magnitude.
- If one feature (like income) has a large range, it can unfairly influence predictions.
- Normalizing ensures **all features contribute equally**.

2. Faster Training

- Models that use gradient descent (like **Logistic Regression, Neural Networks**) converge **faster** when features are scaled.
- Prevents issues like **vanishing or exploding gradients**.

3. Easier Interpretation & Visualization

- Scaled features make it easier to **visualize relationships** between variables.
- Charts and comparisons become clearer.

Example: Housing Price Prediction

Feature	Range
Square Footage	500 – 5000
Bedrooms	1 – 5
Distance to Market	0.1 – 10 miles

Problem:

Without normalization, square footage (large scale) dominates. The model **ignores smaller scale features** like bedrooms or distance.

Solution:

Normalize all features.

Now each one has equal importance in training → **better, fairer model.**

What is Scaling?

Scaling is a broader term that encompasses both normalization and standardization. While normalization aims for a specific range (0-1), scaling adjusts the spread or variability of your data.

Scaling vs. Normalization

Aspect	Scaling (Standardization)	Normalization (Min-Max Scaling)
Definition	Transforms data to have mean = 0 and std deviation = 1	Transforms data to a fixed range (usually 0 to 1)
Formula	$z = \frac{x - \mu}{\sigma}$	$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}$
Resulting Range	Mean = 0, Std Dev = 1 (range not fixed)	Range between 0 and 1 (or custom range)
Sensitive to	Less sensitive to outliers	Highly sensitive to outliers
When to Use	Algorithms that assume normal distribution (e.g., SVM, LR, PCA)	Algorithms that don't assume distribution (e.g., KNN, NN)
Common Tool	StandardScaler from sklearn	MinMaxScaler from sklearn

Types of Normalization/Scaling Techniques

- **Min-Max Normalization**
- **Max-Abs Normalization**
- **Mean Normalization**
- **Log Scaling**
- **Z-Score Normalization (Standardization)**
- **Standardization**
- **Feature Scaling**
- **Dimensionality Reduction**
- **Categorical Scaling**
- **Robust Normalization (Robust Scaling)**

Min-Max Normalization

- Min-Max Normalization also known **as rescaling**, is a popular normalization technique that rescales the data to a common range, usually between **0 and 1**.

- **Formula :**

$$X_{\text{norm}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

Iris Dataset

1. The Iris dataset contains measurements of **sepal** and **petal** lengths and widths of 3 iris flower species.

2. Features (Before Normalization):

Feature Name	Description
sepal length (cm)	Length of the sepal
sepal width (cm)	Width of the sepal
petal length (cm)	Length of the petal
petal width (cm)	Width of the petal

Suppose we have a few rows from the Iris dataset:

Sample	Sepal Length (cm)
A	5.1
B	4.9
C	6.7
D	5.6
E	7.0

3. compute Min-Max Normalization:

- Min = 4.9, Max = 7.0

$$X_{norm} = \frac{X - 4.9}{7.0 - 4.9}$$

Outcome

Sample	Sepal Length	Normalized
A	5.1	0.1667
B	4.9	0.0000
C	6.7	0.8333
D	5.6	0.5833
E	7.0	1.0000

Code

```
#Min Max Normalization
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.preprocessing import MinMaxScaler

# Load real-time Iris dataset
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Initialize the MinMaxScaler
scaler = MinMaxScaler()

# Fit and transform the dataset
normalized_data = scaler.fit_transform(df)
|
# Create a new DataFrame with normalized values
normalized_df = pd.DataFrame(normalized_data, columns=[col + "_norm" for col in df.columns])

# Combine original and normalized data for comparison
result = pd.concat([df, normalized_df], axis=1)

# Display first 5 rows
print(result.head())
```



What can I help you build?

Z-score normalization

- **Z-score normalization, also known as standardization**, transforms data into a standard normal distribution with a mean of 0 and a standard deviation of 1. This technique is useful when the data follows a normal distribution.
- Formula
- x = original value
- μ = mean of the feature
- σ = standard deviation of the feature
- **Best For:** Algorithms that assume normal distribution (e.g., Linear Regression, SVM).

$$X_{\text{standard}} = \frac{X - \mu}{\sigma}$$

Example

- Let's say for **sepal length (cm)** we have:

Sample	Value
1	5.1
2	7.0
3	6.3

Step 1: Calculate Mean (μ)

$$\mu = \frac{5.1 + 7.0 + 6.3}{3} = 6.13$$

Step 2: Calculate Std Dev (σ)

$$\sigma = \sqrt{\frac{(5.1 - 6.13)^2 + (7.0 - 6.13)^2 + (6.3 - 6.13)^2}{3}} \approx 0.78$$

Step 3: Apply Z-Score Formula

Sample	Value	Z-score
1	5.1	$(5.1 - 6.13)/0.78 \approx -1.32$
2	7.0	$(7.0 - 6.13)/0.78 \approx 1.11$
3	6.3	$(6.3 - 6.13)/0.78 \approx 0.22$

Normalization

- Robust Normalization (Robust Scaling)

Uses the **median** and **IQR** (Interquartile Range).

Formula:

$$X_{\text{robust}} = \frac{X - \text{median}}{\text{IQR}}$$

Best For: Datasets with **outliers**.

- Log Normalization

Applies logarithmic scaling:

$$X_{\text{log}} = \log(X)$$

Best For: Data with **exponential growth** or highly skewed distributions.

when to use Normalization (Min-Max Scaling) vs. Standardization (Z-Score Scaling) — with real-time machine learning examples

- **Scenario 1: Image Processing with Neural Networks**
- **Task:** Classifying images (e.g., cats vs. dogs) using a **Convolutional Neural Network (CNN)**.
- **Input Data:** Pixel values ranging from 0 to 255.
- **Preferred Technique: Normalization (Min-Max Scaling)**
- $\text{pixel_norm} = \text{pixel} / 255.0$
- **Why?**
- Neural networks converge faster and perform better when inputs are in a small range (0 to 1).

Scenario 2: Credit Risk Analysis using Logistic Regression

- **Task:** Predicting loan default based on features like age, income, loan amount.
- **Input Data:** Income (10,000–100,000)
- Age (20–60)
- Loan amount (1,000–500,000)
- **Preferred Technique: Standardization (Z-Score Scaling)**
- **Why?**
 - Logistic Regression assumes **normally distributed features**. Standardization centers data (mean = 0), making model training stable.

Scenario 3: Customer Segmentation using K-Means Clustering

- **Task:** Group customers based on spending habits and visit frequency.
- **Input Data:**
 - Monthly spend (₹500–₹50,000)
 - Number of visits per month (1–30)
- **Preferred Technique: Normalization (Min-Max Scaling)**
- **Why?**
 - K-Means is **distance-based**. If data is not scaled, high-range features dominate clustering.

Feature Selection and Filtering

- **Filter Methods**
- **Wrapper Methods**
- **Embedded Methods**

Method Type	Technique	Description	When to Use	Pros	Cons	Example Use Case
Filter	Correlation Matrix	Removes features with high inter-correlation (redundant info)	Continuous features, multicollinearity suspected	Fast, model-agnostic	Ignores target variable	Financial forecasting (remove correlated stock indicators)
Filter	Chi-Square Test	Tests statistical dependence between categorical features and target	Categorical data, classification tasks	Simple, effective	Only for categorical vars	Customer churn prediction
Filter	Mutual Information	Measures mutual dependency between feature and target	Mixed-type data	Captures nonlinear relationships	May be hard to interpret	Text classification (word relevance)
Filter	ANOVA F-test	Compares means of numeric features across categories	Categorical target, numeric features	Good for pre-analysis	Assumes normal distribution	Cancer detection (gene expression data)

Method Type	Technique	Description	When to Use	Pros	Cons	Example Use Case
Wrapper	Forward Selection	Starts with no features, adds one at a time based on model performance	Small feature sets, high interpretability needed	Finds best small subset	Expensive for large sets	Sensor fault diagnosis
Wrapper	Backward Elimination	Starts with all features, removes least useful one by one	Medium-sized datasets	More accurate than filter	Computationally costly	Student performance prediction
Wrapper	Recursive Feature Elimination (RFE)	Recursively removes least important features using model weights	When model has clear feature importance metric	Works with most models	Slow, may overfit	Image classification (pixel-level selection)

Method Type	Technique	Description	When to Use	Pros	Cons	Example Use Case
Embedded	Lasso Regression (L1)	Penalizes less important features, shrinking some to zero	When linear model is appropriate	Simultaneous training + selection	Biased if features are correlated	Predicting housing prices
Embedded	Elastic Net	Combines L1 and L2 for balanced shrinkage	When multicollinearity exists	Handles correlated features	Requires tuning two parameters	Biomedical signal processing
Embedded	Tree-Based Models (e.g., Random Forest)	Use feature importance scores from decision trees	When interpretability and performance matter	Captures nonlinearity, interactions	Bias toward high-cardinality features	Credit scoring, fraud detection
Embedded	XGBoost/LightGBM Feature Importance	Gradient boosting trees give ranked importance	For high-performance models	Very accurate	Sensitive to tuning and imbalance	Insurance claim risk modeling

Filter Method

- Variance Threshold

- Removes features with **low or zero variance** because they do not provide useful information for prediction.
- A feature with the same value for almost all samples does not help the model.

Formula:

$$\text{Variance}(X) = \frac{\sum (X_i - \bar{X})^2}{n}$$

- If variance < threshold → drop the feature.

x_i is the individual value of the feature for the i -th observation.

$\bar{x}$ is the sample mean of the feature.

n is the number of observations.

Example

This technique is unsupervised, meaning it does not consider the target labels when selecting features. It's most effective as a first-pass filter before applying more complex methods.

Steps-by-step Working

- The process of using the Variance Threshold method involves the following steps:
- Calculate variance for each feature in the dataset.
- Compare each variance to the predefined threshold.
- Discard features with variance below the threshold.
- Retain features with sufficient variability.

Dataset (Predict Pass/Fail):

Hours_Studied	Attendance	Fav_Color	Pass_Fail
2	40	Red	0
3	50	Red	0
4	60	Red	1
5	70	Red	1
6	80	Red	1

- Fav_Color has the same value ("Red") for all rows → variance = 0 → drop it.

Use Cases

- **Text Analytics (Bag of Words / TF-IDF)**
 - Many words appear in **all documents** (e.g., “the”, “and”) → variance is **zero or very low**.
 - Variance Threshold helps **remove these uninformative features**.
- **Sensor Data / IoT**
 - If a sensor reports **constant readings**, the feature does not affect prediction.
 - Example: A temperature sensor always reading 25°C adds no value.
- **Finance / Stock Prediction**
 - Removing columns like **Account_Type** if 99% of users have the same type.
- **Image Processing**
 - Pixels that are **always the same color** across images are uninformative.

Applications

- **Spam Email Classification** → Remove words with zero variance.
- **Credit Risk Modeling** → Remove constant features like a product type if all users have the same product.
- **IoT Fault Detection** → Ignore sensors that never vary.

Chi-Square Test (χ^2 Test)

- Measures **dependence between categorical features and the target**.
- Checks if a feature and the target are **independent**.
- It is a non-parametric statistical test meaning it doesn't follow normal distribution.
- High Chi-square score $\rightarrow$ strong relationship.

Formula:

$$\chi^2 = \sum \frac{(O - E)^2}{E}$$

- **O** = Observed frequency
- **E** = Expected frequency

Example

Chi-square test in Machine Learning

		
	Female	Male
Without graduation	6	7
College	13	16
Bachelor's degree	16	15
Master's degree	8	11
Total	43	49

Is there a relationship between gender and the highest level of education?

↓

Chi² - Test

Example of Chi-square test

Ref :
<https://www.geeksforgeeks.org/machine-learning/chi-square-test-in-Data-Science-and-Data-Analytics/>

Example

- We want to know if **Study_Hours_Category** (Low/Medium/High) affects **Pass/Fail**:

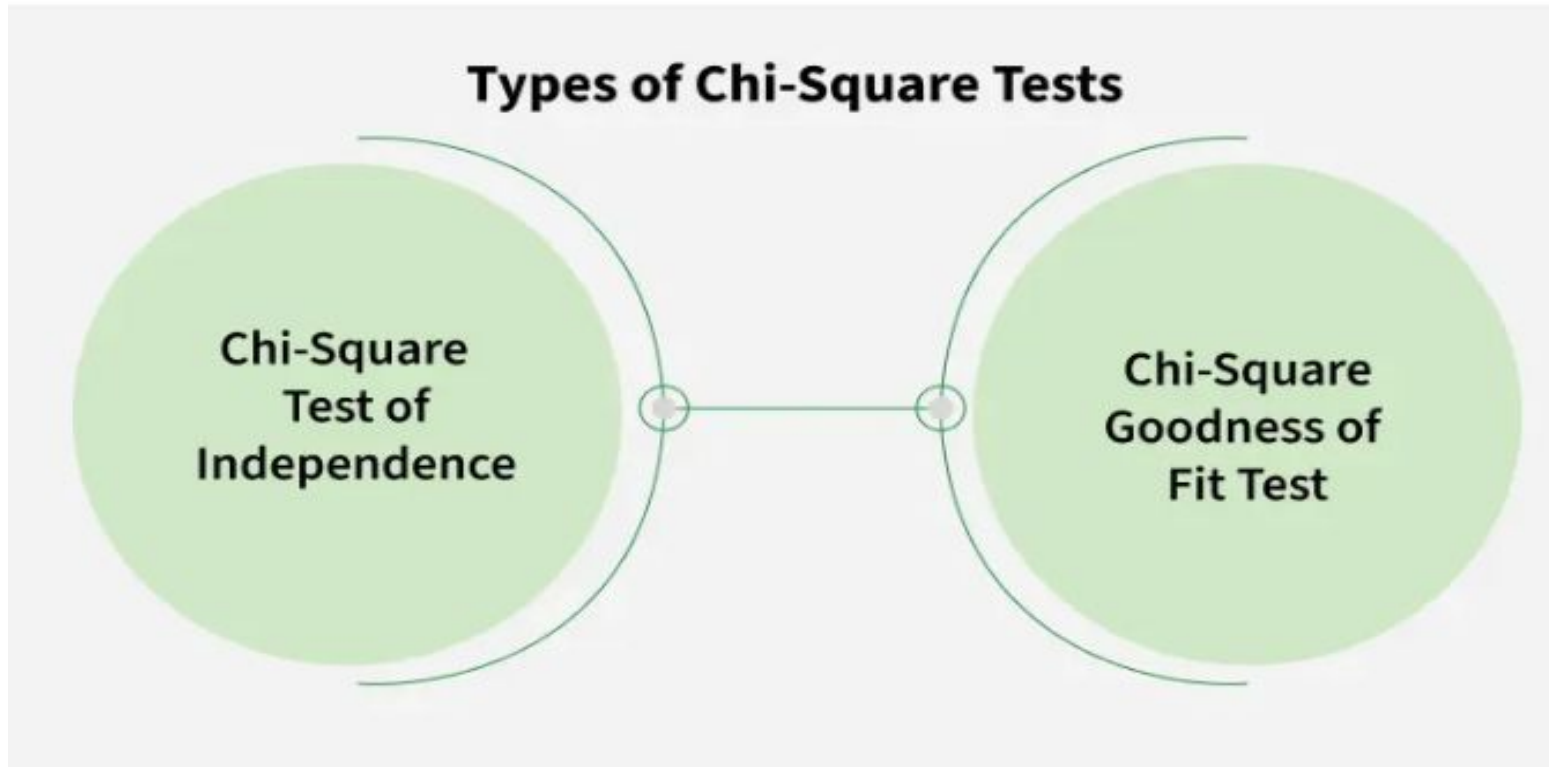
sample

Study_Hours	Category	Pass_Fail
2	Low	0
3	Medium	0
5	High	1
6	High	1
1	Low	0

Create a contingency table

Category	Pass=0	Pass=1
Low	2	0
Medium	1	0
High	0	2

Types



Types of chi-square tests

Use case

- **Customer Segmentation (Retail/Marketing)**
 - Feature: Membership_Type (Silver/Gold/Platinum)
 - Target: Purchased (Yes/No)
 - Use Chi-square to see which **membership levels influence purchase**.
- **Healthcare**
 - Feature: Smoker (Yes/No), Exercise_Level (Low/Medium/High)
 - Target: Has_Disease (Yes/No)
 - Chi-square shows which habits are statistically related to disease occurrence.
- **Banking and Loan Approval**
 - Feature: Occupation_Type
 - Target: Loan_Aproved (Yes/No)
 - Identifies categorical features most relevant for approval prediction.

Application

- **Fraud Detection** → Relationship between Transaction_Type and Fraud_Flag.
- **Market Basket Analysis** → Association between product category and purchase behavior.
- **Medical Research** → Identify categorical risk factors for a disease.

ANOVA F-Test (Analysis of Variance)

- Used when **feature is continuous** and **target is categorical (classification)**.
- Measures how much the **means of the feature values differ between classes**.
- **High F-score → feature is important.**

Formula for F-value:

$$F = \frac{\text{Variance Between Groups}}{\text{Variance Within Groups}}$$

Example

Feature: `Hours_Studied` (Continuous)

Target: `Pass_Fail` (Categorical)

Hours_Studied	Pass_Fail
2	0
3	0
5	1
6	1
4	1

- **Class 0 Mean = $(2+3)/2 = 2.5$**
- **Class 1 Mean = $(5+6+4)/3 = 5$**
- Big difference → High F-value → Good feature
- **`from sklearn.feature_selection import f_classif`**

Use Case

- **Medical Diagnosis**

- Feature: Cholesterol_Level (continuous)
- Target: Disease_Status (Positive/Negative)
- ANOVA F-test finds which lab results **best separate healthy vs unhealthy patients.**

- **Student Performance Prediction**

- Feature: Hours_Studied (continuous)
- Target: Pass/Fail
- ANOVA shows if **study hours have a significant impact** on passing.

- **Finance**

- Feature: Annual_Income (continuous)
- Target: Customer_Tier (Low/Medium/High)
- ANOVA helps determine which financial attributes **differentiate customer segments.**

Application

- **Healthcare** → Lab test values vs disease outcomes.
- **Manufacturing** → Continuous machine metrics vs defect categories.
- **Education Analytics** → Study hours vs exam results.
- **Finance** → Income or spending patterns vs customer category.

Principal Component Analysis

PCA

- **PCA (Principal Component Analysis)** is a dimensionality reduction technique used in data analysis **and** unsupervised machine learning.
- It helps you to reduce the number of features in a dataset while keeping the most important information. The idea of PCA is simple: reduce the number of variables of a data set, while preserving as much information as possible.
- It helps to remove redundancy, improve computational efficiency and make data easier to visualize and analyze especially when dealing with high-dimensional data.

Principal Component Analysis

- **Key Idea:**
- PCA identifies **principal components** (new axes) that capture the **maximum variance** in the data.
- These components are **linear combinations of original features**.
- The **first principal component (PC1)** captures the most variation, followed by the **second principal component (PC2)**, and so on.
- **Example:**
Suppose we have a dataset with 100 features (columns). Many may be correlated. PCA can reduce it to, say, 10 principal components that explain 90–95% of the variance, simplifying analysis without losing much information.

Uses of PCA

- **Dimensionality Reduction:**

- Reduce the number of features for faster training and visualization.

- **Noise Reduction:**

- By keeping only major components, PCA can remove random noise from data.

- **Feature Extraction:**

- It can create new, meaningful features (principal components) for ML models.

- **Data Visualization:**

- High-dimensional data (e.g., 50+ features) can be visualized in 2D or 3D using PCA.

Applications of PCA

- PCA is widely used in **machine learning, statistics, and signal processing**, including:
- **Image Compression and Recognition**
 - Example: Reducing pixel dimensions in face recognition (Eigenfaces).
- **Finance**
 - Stock market analysis to identify factors affecting price movements.
- **Medical Diagnosis**
 - Reduce complexity in high-dimensional data like gene expression or MRI scans.
- **Speech and Signal Processing**
 - Noise removal and feature extraction.
- **Recommendation Systems**
 - Reduce sparse user-item matrices for efficient processing.
- **Exploratory Data Analysis (EDA)**
 - Understand the structure and relationships in high-dimensional data.

Use Case Example

- **Face Recognition (Eigenfaces Approach):**

- Each face image (100x100 pixels) = 10,000-dimensional vector.
- PCA reduces dimensions to ~100–200 principal components (Eigenfaces).
- These components capture the essential facial features, improving model efficiency while reducing storage and computation.

- **Natural Language Processing (NLP)**

- **Text Feature Reduction**

- Convert large sparse word embeddings (TF-IDF vectors) into fewer meaningful components.

- **Topic Modeling (as a preprocessing step)**

- PCA reduces dimensions before applying clustering or LDA for topic discovery.

- **Sentiment Analysis**

- Helps compress large text feature spaces into dense, interpretable components.

How Principal Component Analysis Works

- **Step 1: Standardize the Data**

- PCA is **sensitive to the scale** of features.
- Standardize each feature to have **mean = 0** and **standard deviation = 1**.

- **Example:**

Dataset with 2 features:

Sample	Height (cm)	Weight (kg)
A	160	60
B	165	65
C	170	68
D	175	72

$$Z = \frac{X - \mu}{\sigma}$$

Standardize → Convert to z-scores (scaled data).

How Principal Component Analysis Works

Step 2: Compute Covariance Matrix

- Find how variables vary together:

$$Cov(X, Y) = \frac{\sum (X_i - \bar{X})(Y_i - \bar{Y})}{n - 1}$$

- A 2D dataset $\rightarrow$ 2×2 covariance matrix:

$$Cov = \begin{bmatrix} Var(X) & Cov(X, Y) \\ Cov(Y, X) & Var(Y) \end{bmatrix}$$

Step 3: Compute Eigenvalues and Eigenvectors

- Eigenvectors** = directions of principal components
- Eigenvalues** = amount of variance each component captures

Example:

- PC1 $\rightarrow$ Eigenvalue = 1.6 (captures 80% variance)
- PC2 $\rightarrow$ Eigenvalue = 0.4 (captures 20% variance)

How Principal Component Analysis Works

Step 4: Sort and Select Principal Components

- Sort components by eigenvalue (variance explained).
 - Decide how many components to keep (e.g., 95% variance).
-

Step 5: Transform Data to New Subspace

- Project original standardized data onto selected principal components.

Example:

- 2D data (Height, Weight) → 1D (PC1)
- New dataset shows most variance in one feature instead of two.

How Principal Component Analysis Works

Step 6: (Optional) Reconstruct or Visualize

- Use first k principal components to visualize in 2D or 3D.
 - Useful for data compression or clustering.
-

Illustrative Example

1. **Input Data:** Students' scores in **Math** and **Physics** (highly correlated).
2. **Standardize** scores $\rightarrow$ Mean 0, SD 1.
3. **Covariance Matrix** $\rightarrow$ Shows strong positive correlation.
4. **Eigenvalues/Eigenvectors** $\rightarrow$
 - PC1 = "Overall performance" (captures 90% variance)
 - PC2 = "Subject difference" (captures 10% variance)
5. **Transform Data** $\rightarrow$ 2 scores $\rightarrow$ 1 principal component
6. **Use Case** $\rightarrow$ Reduce features for clustering or visualization.

What is Non-Negative Matrix Factorization (NMF)?

- **Non-Negative Matrix Factorization (NMF)** is a **dimensionality reduction and feature extraction technique** that factorizes a **non-negative matrix** into two smaller non-negative matrices.

Original matrix $V \approx W \times H$

V = Original data (all values ≥ 0)

W = Basis matrix (features/components)

H = Coefficient matrix (weights for each component)

- **Key Rule:**

All matrices involved must have **no negative elements**.

Matrix Decomposition and Representation in NMF

- For a matrix A of dimensions $m \times n$ where each element is ≥ 0 NMF factorizes it into two matrices W and H with dimensions $m \times k$ and $k \times n$ respectively where both matrices contain only non-negative elements:

$$A_{m \times n} \approx W_{m \times k} H_{k \times n}$$

where:

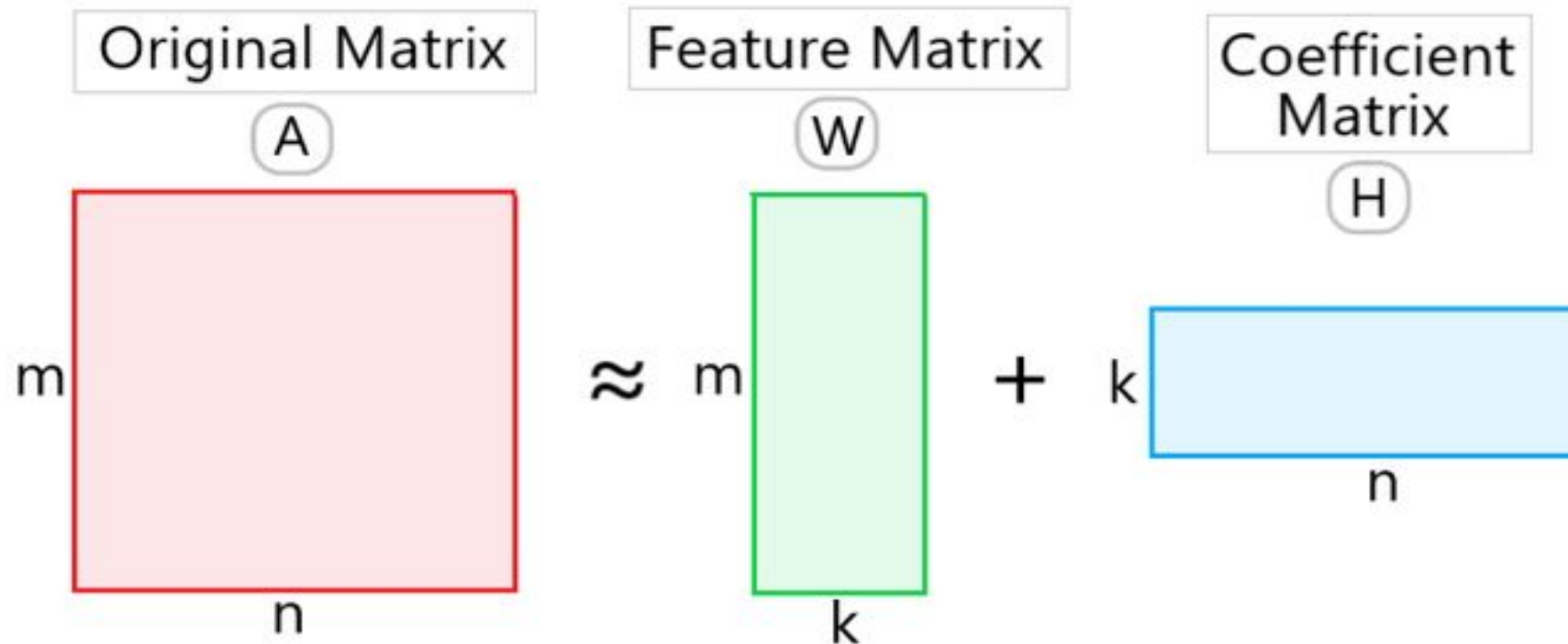
$A \rightarrow$ Original input matrix (a linear combination of W and H)

$W \rightarrow$ Feature matrix (basis components)

$H \rightarrow$ Coefficient matrix (weights associated with W)

$k \rightarrow$ Rank (dimensionality of the reduced representation where $k \leq \min(m, n)$)

. How NMF Works (Intuition)



. How NMF Works (Intuition)

Suppose we have a **document-term matrix** in NLP (Natural Language Processing):

- V = Documents $\times$ Words matrix (counts of words in documents)

NMF factorizes it as:

$$V_{(documents \times words)} \approx W_{(documents \times topics)} \times H_{(topics \times words)}$$

- W (Documents $\times$ Topics) $\rightarrow$ How much each topic is present in each document
- H (Topics $\times$ Words) $\rightarrow$ Which words define each topic

This helps in **topic modeling** without requiring negative numbers (since word counts are always ≥ 0).

. How NMF Works (Intuition)

- Step 1: Input Matrix (V)
- Imagine we have **4 documents and 6 words** (term frequency table):

	Word1	Word2	Word3	Word4	Word5	Word6
Doc1	2	3	0	0	1	0
Doc2	3	4	0	0	2	0
Doc3	0	0	4	5	0	3
Doc4	0	0	3	4	0	2

. How NMF Works (Intuition)

Step 2: Factorize into W and H ($k=2$ topics)

W (Documents $\times$ Topics):



	Topic1	Topic2
Doc1	0.8	0.1
Doc2	0.9	0.2
Doc3	0.1	0.7
Doc4	0.2	0.8

H (Topics $\times$ Words):



	Word1	Word2	Word3	Word4	Word5	Word6
Topic1	0.7	0.8	0.1	0.1	0.5	0.0
Topic2	0.0	0.1	0.8	0.9	0.0	0.7

Interpretation

- Topic1 = Words {1,2,5} → likely about **Subject A**
- Topic2 = Words {3,4,6} → likely about **Subject B**

$$\begin{array}{c} \text{Original Matrix (V)} \\ \text{Documents} \times \text{Words} \end{array} \approx \begin{array}{c} \text{Factorization} \\ (\text{Documents} \times \text{Topics}) \times (\text{Topics} \times \text{Words}) \end{array}$$

Doc1 →	$\begin{bmatrix} 2 & 3 & 0 & 0 & 1 & 0 \end{bmatrix}$	$\approx$	$\begin{bmatrix} 0.8 & 0.1 \end{bmatrix}$	$\times$	$\begin{bmatrix} 0.7 & 0.8 \dots \end{bmatrix}$
Doc2 →	$\begin{bmatrix} 3 & 4 & 0 & 0 & 2 & 0 \end{bmatrix}$		$\begin{bmatrix} 0.9 & 0.2 \end{bmatrix}$		$\begin{bmatrix} \dots \end{bmatrix}$
Doc3 →	$\begin{bmatrix} 0 & 0 & 4 & 5 & 0 & 3 \end{bmatrix}$		$\begin{bmatrix} 0.1 & 0.7 \end{bmatrix}$		$\begin{bmatrix} \dots \end{bmatrix}$
Doc4 →	$\begin{bmatrix} 0 & 0 & 3 & 4 & 0 & 2 \end{bmatrix}$		$\begin{bmatrix} 0.2 & 0.8 \end{bmatrix}$		$\begin{bmatrix} \dots \end{bmatrix}$

Applications of NMF

- **Topic Modeling in NLP**

- Automatically discover topics in large text collections (e.g., news articles).

- **Image Processing**

- Decompose images into parts/features (e.g., facial recognition using parts like eyes, nose, mouth).

- **Recommender Systems**

- Movie recommendation based on user-item interaction matrices.

- **Bioinformatics**

- Analyze gene expression data for identifying hidden patterns.

Task

Work on Difference between PCA AND
NMG

Sparse Principal Component Analysis (Sparse PCA)

- **What is Sparse PCA?**

- A variant of PCA where the **principal components have many zero (sparse) loadings**.
- This means **each component depends only on a few original features**, improving **interpretability**.
- Standard PCA uses **all features** for each principal component.

- **How it Works**

- Standardize the data.
- Compute principal components with an added **sparsity constraint (L1 regularization)**.
- Components highlight **only the most important features**.

Sparse Principal Component Analysis (Sparse PCA)

- **Example**
- In **genomic data** (10,000+ genes):
 - Standard PCA → PC1 is a combination of all genes.
 - Sparse PCA → PC1 is influenced by **only the top 50 genes**, making it easier to identify meaningful biological patterns.
- **Use Cases**
- **Bioinformatics** → Gene selection for disease classification.
- **Finance** → Identify the most influential few stocks driving market trends.
- **High-dimensional data analysis** → Feature selection + dimensionality reduction.

Kernel PCA (KPCA)

- **What is Kernel PCA?**
- An **extension of PCA for non-linear data.**
- Uses the **kernel trick** to project data into a **higher-dimensional space** where it becomes linearly separable.
- Standard PCA only works with **linear patterns.**

Kernel PCA (KPCA)

- **Example**
- **Circular data** in 2D cannot be separated with standard PCA.
- Kernel PCA (e.g., using an RBF kernel) maps the circle into a linearly separable feature space.
- **Use Cases**
- **Image denoising** → Capture non-linear patterns in pixel intensities.
- **Anomaly detection** → Detect non-linear deviations in sensor data.
- **Pattern recognition** → Non-linear data structures like handwriting and facial recognition.

Atom Extraction and Dictionary Learning

- **What is Dictionary Learning?**
- A **feature learning technique** that represents data as a **sparse combination of basis vectors** called **atoms**.
- Goal: Learn a **dictionary of atoms** that can efficiently represent the data.
- **Key Idea**
- Original Data : $X \approx D \times A$
 - D = Dictionary (atoms)
 - A = Sparse coefficients

Atom Extraction

- **Atoms** are the **basic building blocks** (small patterns) extracted from the data.
- Each data sample is a **sparse combination** of a few atoms.
- **Example**
- **Image patches for denoising:**
 - Extract small **8×8 pixel patches** from images.
 - Learn dictionary atoms that represent edges and textures.
 - Reconstruct image using a few atoms → **Noise reduced.**

Use Cases

- **Image Denoising & Compression** (K-SVD algorithm).
- **Speech Signal Processing** → Extract phoneme patterns.
- **Anomaly Detection** → Sparse representation highlights unusual events.
- **Face Recognition** → Represent faces as combinations of dictionary atoms.

Summary

Technique	Key Idea	Use Case
Sparse PCA	PCA with sparse loadings	Gene selection, feature reduction
Kernel PCA	PCA in high-dimensional kernel space	Non-linear patterns, anomaly detection
Dictionary Learning	Learn atoms to represent data sparsely	Image denoising, face recognition
Atom Extraction	Extract basic building blocks (atoms)	Speech, image compression

CA	Type of Assessment	Date	Mode of Conduction	Marks	Syllabus
CA1	Home assignment	11-07-26	offline	10	unit 1,Unit 2 (some part)
CA2	Springboard Course	14-8-26	offline	10	Unit2,unit3(some part)
CA3	Research Article Review : Phase 1	12-9-26	Online/offline	10	Unit 3,Unit 4 and Unit 5 (some part)
CA4	Implementation: Phase 2	01-10-26	offline	10	All units